Duplicate Finder – Quick Interview & Project Notes

Python-based Duplicate File Finder

1. Interview-Ready Project Explanation

“I developed a Python-based Duplicate File Finder that detects duplicate files inside a folder and its subfolders.

The main problem I wanted to solve was that users can have multiple copies of the same file, which unnecessarily consumes storage space.

I used Python's `os.walk()` to recursively scan the folder and all its subfolders. For each file, I read the contents in chunks and generated an MD5 hash using Python's `hashlib`. Files having the same hash were grouped together using a dictionary with `defaultdict(list)`.

I also calculated the wasted storage space by keeping one copy conceptually and calculating the size occupied by the remaining duplicate copies.

During testing, I created different test cases including duplicates in the same folder, duplicates in different subfolders, unique files, and larger files. For example, with four identical 5 MB files, the program correctly reported **15 MB of wasted space**, because one copy can be retained and the other three copies are redundant.

Through this project, I learned **file handling, directory traversal, hashing, dictionaries, exception handling, and memory-efficient file processing using chunks**. I also learned how to test a program with different edge cases rather than just checking whether the code runs.

One important thing I learned is that I'm comparing the **file contents**, not the filenames. So even if two files have different names or are in different folders, they can still be identified as duplicates if their contents are identical.”

2. What Exactly Did You Learn?

Topic	What I learned
`os.walk()`	How to recursively traverse directories and subdirectories.
`hashlib`	How hashing can be used to compare file contents efficiently.
MD5	Same content produces the same hash, allowing files to be grouped.
`defaultdict`	How to group multiple file paths under the same hash.
File handling	Opening and reading binary files using `"rb"`.
Chunk processing	Reading large files in small chunks instead of loading the entire file into memory.
Exception handling	Handling inaccessible or deleted files with `try/except`.
Algorithm thinking	Turning a real-world storage problem into a systematic solution.
Testing	Testing nested folders, unique files, different filenames, and large files.

3. Why Did You Use Hashing?

“Comparing every file directly with every other file would be inefficient. Instead, I generate a hash representing the file's contents. Files with the same hash are potential duplicates, so I can group them efficiently. I also process files in chunks, which prevents large files from consuming too much memory.”

4. Why MD5? Isn't MD5 Insecure?

“Yes, MD5 is not suitable for security or cryptographic applications because collisions can be deliberately generated. For this project, I used it as a fast content fingerprint for duplicate detection. For a production version, I could use SHA-256 or combine file size and hashing to make the detection more robust.”

5. Important Technical Concepts

Concept	Simple explanation
Chunks	A chunk is a small portion of a file. The project reads files in 8 KB chunks so large files do not need to be loaded completely into memory.
MD5 hash	MD5 (Message-Digest Algorithm 5) produces a 128-bit hash, normally represented by 32 hexadecimal characters. Identical content normally produces the same hash. MD5 is not recommended for security-sensitive applications.
Hash	A hash is a digital fingerprint of data. In this project, it is used as a dictionary key to group files with matching content.
How files are grouped	A defaultdict(list) maps each hash to a list of file paths. Each file is appended to the list for its hash. Groups containing more than one path are reported as duplicate sets.
rb	"rb" means read binary mode. It reads the exact bytes of a file and works for text, PDFs, images, videos, ZIPs and other binary formats.
SHA-256	SHA-256 (Secure Hash Algorithm 256-bit) creates a 256-bit hash represented by 64 hexadecimal characters. It provides much stronger collision resistance than MD5.
Hash collision	A collision occurs when different inputs produce the same hash. MD5 has known collision weaknesses; SHA-256 is much more resistant.
MD5 vs SHA-256	MD5 is faster and smaller but cryptographically weak. SHA-256 is stronger and preferred when stronger collision resistance is required.

6. Common Interview Questions & Answers

Q: What problem did you solve?

A: Users can have multiple copies of the same file, wasting storage. The project detects content-identical files and estimates redundant storage.

Q: Why use hashing?

A: It provides a compact fingerprint of file contents and makes grouping potential duplicates practical.

Q: Why use os.walk()?

A: It recursively scans the selected folder and all its subfolders.

Q: Why use defaultdict(list)?

A: It makes it easy to map one hash to multiple file paths.

Q: Why use rb?

A: It reads exact binary bytes and supports many file types, not only text files.

Q: What are chunks?

A: Small pieces of a file read one at a time to reduce memory usage.

Q: What is MD5?

A: A 128-bit hashing algorithm that creates a 32-character hexadecimal fingerprint.

Q: What is SHA-256?

A: A 256-bit hashing algorithm that creates a 64-character hexadecimal fingerprint with much stronger collision resistance.

Q: Does the program compare filenames?

A: No. It compares file contents through their hashes, so different names can still be duplicates.

Q: How is wasted space calculated?

A: For a duplicate group, one copy is conceptually retained and the remaining copies are counted as wasted: file size × (number of copies – 1).

Q: What happens if a file cannot be read?

A: PermissionError and FileNotFoundError are caught and that file is skipped so the scan can continue.

Q: Does it scan ZIP files directly?

A: No. The current program scans folders. ZIP files must be extracted first unless ZIP handling is added.

Q: Why did you use MD5?

A: It is fast and sufficient for a basic duplicate-content fingerprint. For stronger collision resistance in a production version, SHA-256 would be a better choice.

Q: What is a limitation?

A: The current version reports duplicates but does not provide an interactive delete feature. It could be improved with safe deletion confirmation and stronger optimization.

7. Testing Results

Test	Result
Original test folder with nested duplicates	Passed
ZIP 1 – simple duplicates	Passed
ZIP 2 – nested duplicates	Passed
ZIP 3 – mixed duplicates and unique files	Passed
Large test – four identical 5 MB files	Passed; 15 MB wasted space reported

8. Strong One-Line Project Explanation

“I recursively scan files, read them in binary chunks, generate a content hash, group files with the same hash, report duplicate sets, and calculate the storage occupied by redundant copies.”

9. Future Improvements

• Use SHA-256 instead of MD5 for stronger collision resistance.

• Compare file sizes before hashing to reduce unnecessary work.

• Group files by size first, then hash only files with matching sizes.

• Add safe user confirmation before deleting selected duplicates.

• Add command-line arguments and a cleaner user interface.

• Export duplicate reports to CSV or JSON.

• Add tests for permission errors, empty files, very large files, and symbolic links.